

Phase 1: Problem Understanding and Literature Review

Objective

To translate the clinical medical problem of CKD into a well-defined Machine Learning classification task, and to identify a "research gap" in existing literature that your project will address.

Theory

- **The Clinical Problem:** CKD is a condition where the kidneys gradually lose their ability to filter waste from the blood. It is a "silent killer" — patients often show no symptoms until stage 4 or 5, by which point dialysis or a transplant is required. Early stages (1-3) can be managed with diet and basic medication if detected.
- **The ML Translation:** We are building a **Supervised Binary Classification** model.
 - **Input (\$X\$):** Patient medical metrics (blood pressure, serum creatinine, hemoglobin, etc.).
 - **Output (\$y\$):** A binary label (1 = CKD positive, 0 = CKD negative).
- **Evaluation Metric Shift:** In medical AI, **Accuracy is dangerous**. If 90% of a dataset does not have CKD, a "dumb" model that always predicts "No CKD" is 90% accurate, but it will let every sick patient die. We must optimize for **Recall** (Sensitivity) to minimize False Negatives.

Why It Is Important

Most existing diagnostic methods require expensive lab setups. By utilizing machine learning, we can identify high-risk patients using cheaper, easily accessible routine blood and urine tests. If deployed as a backend API for a mobile application, it enables preliminary screening at remote community clinics.

Implementation: Defining the Research Gap

Since this has thesis potential, you cannot just replicate what others have done. A literature review involves searching Google Scholar for papers like *"Machine Learning for Chronic Kidney Disease Prediction."*

When reading, look for these common weaknesses (which will become your strengths):

1. **Poor handling of missing data:** Many papers simply delete rows with missing data. In healthcare, missing data is normal. We will use advanced imputation.
2. **Black-box models:** Doctors will not use a model if they do not understand it. We will use Explainable AI (XAI) to show exactly *which* feature caused the prediction.

Note: There is no Python code for this stage. This phase is entirely analytical and conceptual.

Best Practices

- **Understand the Features:** Do not treat the data as just numbers. Research what *Serum Creatinine* or *Specific Gravity* actually mean clinically. It will guide your feature engineering later.
- **Define Success Early:** Decide now that a high Recall and a strong ROC-AUC score are your primary goals.

Common Mistakes

- **Skipping the domain research:** Trying to engineer features without knowing basic kidney biology leads to poor models and disastrous viva defenses.
- **Ignoring Class Imbalance:** Not realizing early on that sick patients and healthy patients might not be equally represented in the data.

Expected Output for this Phase

You should write a 1-2 paragraph **Problem Statement** for your final report or README.

Draft Example: "Chronic Kidney Disease often progresses undetected in resource-constrained regions due to a lack of early screening. This project aims to develop an

interpretable binary classification model capable of predicting early-stage CKD using routine clinical test data. By addressing the challenge of missing clinical data through advanced imputation and employing Explainable AI techniques, this work provides a reliable, transparent screening tool suitable for integration into telehealth applications."

How to Explain It During Viva

Examiner: "Why did you choose CKD over a standard dataset like Iris or House Prices?"

Your Answer: "I wanted to solve a problem with tangible societal impact, particularly for regions with limited healthcare access. CKD is asymptomatic in early stages, making early prediction a life-saving intervention. Unlike standard datasets, medical data is messy, contains critical missing values, and requires high Recall rather than just raw accuracy. It allowed me to demonstrate advanced preprocessing and model explainability, which are essential for real-world AI deployment."

Take a moment to absorb this. Once you are comfortable with the clinical problem and our classification goal, we can open our Jupyter Notebook.

Phase 2: Dataset Collection and Exploratory Data Analysis (EDA)

Objective

To load the raw data, understand its structural integrity (data types, missing values), visualize the target distribution (class imbalance), and uncover relationships between medical features.

Theory

Exploratory Data Analysis (EDA) is like a doctor taking a patient's history and vitals before making a diagnosis. You cannot build a model without knowing the "shape" of your data. We look for three main things:

1. **Data Types:** Are numerical columns accidentally being read as text (strings)?
2. **Missingness:** How much data is missing, and is there a pattern to it?
3. **Distribution:** Is the target variable (CKD vs. Not CKD) balanced?

Why It Is Important

"Garbage in, garbage out." If you feed raw, uninspected data into an algorithm, it will either crash due to missing values or learn the wrong patterns. For example, if 90% of a column is missing, we might need to drop that feature entirely rather than trying to guess the missing values.

Implementation: Python Code

Open a new Jupyter Notebook. Run this code block by block.

Python

```
# 1. Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Set visualization style for professional looking plots
sns.set_theme(style="whitegrid")
```

```
# 2. Dataset Collection (Loading the data)
```

```

# The UCI dataset often represents missing values with '?'.
# We explicitly tell Pandas to treat '?' as NaN (Not a Number) during loading.
url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/chronic_kidney_disease.csv"
# Note: You can also download the CSV from Kaggle/UCI and load it locally:
pd.read_csv('kidney_disease.csv')

df = pd.read_csv(url, na_values=['?'])

# 3. Rename columns for better readability
# The original dataset uses abbreviations (e.g., 'bp' for blood pressure, 'sg' for specific gravity).
# Readable columns make your EDA and final presentations much more professional.
columns_mapping = {
    'bp': 'blood_pressure', 'sg': 'specific_gravity', 'al': 'albumin', 'su': 'sugar',
    'rbc': 'red_blood_cells', 'pc': 'pus_cell', 'pcc': 'pus_cell_clumps', 'ba': 'bacteria',
    'bgr': 'blood_glucose_random', 'bu': 'blood_urea', 'sc': 'serum_creatinine',
    'sod': 'sodium', 'pot': 'potassium', 'hemo': 'hemoglobin', 'pcv': 'packed_cell_volume',
    'wc': 'white_blood_cell_count', 'rc': 'red_blood_cell_count', 'htn': 'hypertension',
    'dm': 'diabetes_mellitus', 'cad': 'coronary_artery_disease', 'appet': 'appetite',
    'pe': 'pedal_edema', 'ane': 'anemia', 'class': 'target'
}
df.rename(columns=columns_mapping, inplace=True)

# Clean target variable (sometimes 'ckd\t' appears due to typos in the raw file)
df['target'] = df['target'].replace({'ckd\t': 'ckd', 'notckd': 'notckd'})
df['target'] = df['target'].map({'ckd': 1, 'notckd': 0}) # Encode target immediately

# 4. Basic Inspection
print("Dataset Shape:", df.shape)
print("\n--- Data Types & Missing Values ---")
print(df.info())

# 5. Visualizing the Target Distribution (Class Imbalance check)
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='target', palette=['#2ecc71', '#e74c3c'])
plt.title("Distribution of CKD vs Not CKD")
plt.xticks(ticks=[0, 1], labels=['Not CKD (0)', 'CKD (1)'])
plt.show()

# 6. Visualizing Missing Data Patterns
# A heatmap allows us to see where data is missing across all rows at once.
plt.figure(figsize=(12, 6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title("Missing Data Heatmap (Yellow = Missing)")
plt.show()

```

```
# 7. Correlation matrix for numerical features
# High correlation between features can cause multicollinearity issues later.
plt.figure(figsize=(12, 8))
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
sns.heatmap(df[numerical_cols].corr(), annot=True, cmap='coolwarm', fmt=".2f",
linewidths=0.5)
plt.title("Correlation Matrix of Numerical Features")
plt.show()
```

Best Practices

- **Handle Missing Values on Read:** Using `na_values=['?']` directly in `pd.read_csv` saves you from writing loops to clean string characters out of numerical columns later.
- **Rename Columns:** Always map obscure clinical abbreviations to full English terms. It makes your code self-documenting.

Common Mistakes

- **Ignoring data types (dtypes):** Because of the '?' characters, Pandas might load numerical columns (like White Blood Cell Count) as object (string) types. If you don't fix this during EDA/Cleaning, math operations (and ML algorithms) will crash.
- **Target Leakage:** Looking at the correlation matrix and seeing a feature that correlates 1.0 with the target. Always ensure your features are data gathered *before* the diagnosis, not the diagnosis itself.

Expected Output

When you run this code, you will observe a few critical things:

1. **Imbalance:** You will likely see more CKD cases than non-CKD cases in this specific UCI dataset (roughly 250 to 150). This tells us we need robust evaluation metrics (like F1-score), not just accuracy.
2. **Missingness Heatmap:** The yellow stripes in the heatmap will show you that certain tests (like Red Blood Cell Count) have a lot of missing data.
3. **Correlations:** You will see strong correlations between variables like hemoglobin and packed_cell_volume.

How to Explain It During Viva

Examiner: "Why did you plot a heatmap for missing values instead of just printing the sum of missing rows?"

Your Answer: "Printing the sum only tells me *how much* data is missing. The heatmap shows me the *pattern* of missingness. For instance, I could see if a specific group of patients was missing a whole batch of lab results simultaneously, which helps me decide whether to impute those values or drop the rows entirely. It gives me clinical context, not just statistical counts."

Run the code in your Jupyter environment, look at the plots, and review the missing values. Let me know when you have successfully executed the EDA and are ready to tackle the messiest part of the project: Data Cleaning.

This is where the real engineering happens. In the industry, data scientists spend about 70% of their time on data cleaning.

If you looked closely at the `df.info()` output from Phase 2, you might have noticed a major problem: columns like `packed_cell_volume` and `white_blood_cell_count` were listed as

object (text) types, even though they should be numbers. This happens because the raw dataset contains hidden typos like `\t43` or `\tyes`.

Let's clean this up scientifically.

Phase 3: Data Cleaning and Preprocessing

Objective

To resolve incorrect data types, remove hidden typos, encode text into machine-readable numbers, and intelligently impute (fill in) missing values without distorting the clinical reality.

Theory

- **Imputation vs. Dropping:** If we drop every row that has a missing value, we will lose almost half our dataset. Instead, we **impute** (guess the missing value based on math).
- **Mode Imputation (Categorical):** For missing text categories (like whether a patient has hypertension), we fill in the most common answer (the mode).
- **KNN Imputation (Numerical):** Instead of just filling missing blood pressure with the average blood pressure of *everyone*, we use **K-Nearest Neighbors (KNN)**. It looks for 5 patients who have similar profiles to our patient and takes the average of *their* blood pressure. This is a highly publishable, research-grade approach.

Why It Is Important

Standard machine learning models (like Logistic Regression or SVM) cannot handle empty cells (NaN) or text ("yes", "abnormal"). The data must be a perfectly clean grid of numbers before training.

Implementation: Python Code

Add this code in a new cell in your Jupyter Notebook, right below the Phase 2 code.

Python

```
from sklearn.impute import KNNImputer
from sklearn.preprocessing import LabelEncoder
```

```
print("Starting Data Cleaning...")
```

```
# 1. Clean invisible typos (tabs and spaces) in categorical columns
```

```
# The dataset has hidden strings like '\tyes', ' yes', or '\tno'
```

```
categorical_cols = df.select_dtypes(include=['object']).columns
```

```
for col in categorical_cols:
```

```
    # Convert to string, strip whitespace and tabs, then replace 'nan' back to actual np.nan
```

```
    df[col] = df[col].astype(str).str.strip()
```

```
    df[col] = df[col].replace('nan', np.nan)
```

```
# 2. Fix Incorrect Data Types
```

```
# These columns are numerical but got parsed as objects due to typos in the raw file
```

```
cols_to_numeric = ['packed_cell_volume', 'white_blood_cell_count', 'red_blood_cell_count']
```

```
for col in cols_to_numeric:
```

```
    # errors='coerce' forces any unreadable text (like '?') to become NaN
```

```
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
# 3. Handle Missing Values (Imputation)
```

```

# Update our lists of categorical and numerical columns after the dtype fix
categorical_cols = df.select_dtypes(include=['object']).columns
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Do NOT impute the target variable! Drop it from the numerical list temporarily.
if 'target' in numerical_cols:
    numerical_cols = numerical_cols.drop('target')

# A. Categorical Imputation: Fill missing with Mode (most frequent value)
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# B. Numerical Imputation: Use KNN Imputer (Research-level)
print("Applying KNN Imputation for numerical features...")
knn_imputer = KNNImputer(n_neighbors=5)
df[numerical_cols] = knn_imputer.fit_transform(df[numerical_cols])

# 4. Encode Categorical Variables (Label Encoding)
# Machine Learning models need numbers, not text like 'normal' or 'abnormal'
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le # We save the encoder for deployment later!

# 5. Final Sanity Check
print("\n--- Data Types After Cleaning ---")
print(df.info())
print("\n--- Total Missing Values After Cleaning ---")
print(df.isnull().sum().sum()) # Should print 0

```

Best Practices

- **Save your Encoders:** Notice how I saved the LabelEncoder in a dictionary (label_encoders[col] = le). When you deploy this as a web app later, a user will input "yes" for diabetes, and you will need this saved encoder to translate it back to the exact number the model expects.
- **Never Impute the Target:** We explicitly removed target from the imputation list. You never guess a patient's diagnosis—if the diagnosis is missing, you drop the row (luckily, our dataset has all targets).

Common Mistakes

- **Data Leakage:** Using simple Mean Imputation can sometimes introduce bias. KNN is safer for medical data because it relies on patient similarity.
- **Missing .astype(str) before stripping:** If you try to run .str.strip() on a column that already has NaN (float) values, Python throws an error.

Expected Output

When you run this, df.info() will show 400 entries, completely non-null, and every single column will be an int64 or float64. The final missing value count will be strictly 0.

How to Explain It During Viva

Examiner: "Why did you use KNN Imputer instead of just taking the average (mean) of the column to fill missing values?"

Your Answer: "Clinical data is highly individualized. If a patient is missing their hemoglobin level, filling it with the global average ignores their other health conditions. KNN Imputer calculates the multidimensional distance between patients. It finds 5 patients with very similar blood pressure, sugar, and age profiles, and averages *their* hemoglobin. This preserves the underlying clinical variance and results in a much more accurate predictive model."

Run the code block. Once you confirm the dataset is 100% clean and numeric, we will move to **Phase 4: Feature Selection and Data Splitting**, where we separate the signal from the noise.

Phase 4: Data Splitting, Scaling, and Feature Selection

Excellent. A clean dataset is a researcher's best friend.

Now, we are at a critical juncture where many beginners make a fundamental error called **Data Leakage**. If you don't handle this phase correctly, your model will look like a genius in the lab, but fail completely when deployed in a real hospital.

Let's do this the right way.

Phase 4: Data Splitting, Scaling, and Feature Selection

Objective

To securely isolate a "test set" that the model has never seen, normalize the numerical ranges of our features, and identify the most biologically significant predictors of CKD.

Theory

- **Data Splitting:** We divide the data into a Training Set (80%) and a Test Set (20%). Because we have class imbalance (more CKD patients than non-CKD), we must use **Stratified Splitting**. This ensures the 80/20 split maintains the exact same ratio of sick-to-healthy patients in both sets.
- **Feature Scaling:** Look at your data. blood_pressure might be 120, while specific_gravity is 1.015, and red_blood_cell_count is 5,200,000. Machine learning algorithms (like SVM or KNN) calculate distance. If we don't scale these numbers to a standard range (e.g., mean of 0, standard deviation of 1), the algorithm will think Red Blood Cells are a million times more important than Specific Gravity just because the number is bigger.
- **Feature Selection:** Not all 24 features are useful. We will use a Random Forest algorithm to rank the features by their mathematical importance. This provides a beautiful graph for your thesis and allows us to potentially drop useless features later to make the model lighter.

Why It Is Important (The Golden Rule)

You MUST split the data BEFORE you scale it.

If you calculate the average blood pressure of the *entire* dataset and use that to scale the features, your training data has secretly learned mathematical information about the test data. This is Data Leakage. The test set must remain locked away and completely unseen until the very end.

Implementation: Python Code

Add this into your next Jupyter cell:

Python

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
```

```
print("Starting Splitting, Scaling, and Feature Selection...")
```

```
# 1. Separate Features (X) and Target (y)
```

```
X = df.drop('target', axis=1)
```

```
y = df['target']
```

```
# 2. Train-Test Split (Stratified)
```

```
# test_size=0.2 means 20% for testing, 80% for training.
```

```
# random_state=42 ensures reproducibility (you get the exact same split every time you run it)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
print(f"Training data shape: {X_train.shape}")
```

```
print(f"Testing data shape: {X_test.shape}")
```

```
# 3. Feature Scaling (Preventing Data Leakage)
```

```
scaler = StandardScaler()
```

```
# FIT on the training data ONLY, then transform it
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
# ONLY transform the test data (do NOT fit it)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# Convert NumPy arrays back to Pandas DataFrames to keep column names
```

```
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns)
```

```
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns)
```

```
# 4. Feature Selection (Using Random Forest Importance)
```

```
# This model is just to calculate feature importance for our research
```

```
rf_selector = RandomForestClassifier(random_state=42, n_estimators=100)
```

```
rf_selector.fit(X_train_scaled, y_train)
```

```
# Create a DataFrame for visualization
```



```
feature_importances = pd.Series(rf_selector.feature_importances_, index=X.columns)
```

```
# Plotting the Top 15 Most Important Features
```

```
plt.figure(figsize=(10, 8))
```

```
feature_importances.nlargest(15).plot(kind='barh', color='#2c3e50')
```

```
plt.title("Top 15 Most Important Clinical Features for CKD Prediction")
```

```
plt.xlabel("Importance Score (Gini Importance)")
```

```
plt.ylabel("Features")
```

```
plt.gca().invert_yaxis() # Highest importance at the top
```

```
plt.show()
```

```
# 5. Save the scaled data for the next phase (Optional but good practice in Jupyter)
```

```
# We will use X_train_scaled and X_test_scaled going forward.
```

Best Practices

- **Save your Scaler:** Just like the LabelEncoder in Phase 3, if you deploy this as a mobile app backend, a new patient's blood pressure of "130" must be scaled using this exact scaler before the model can predict on it.
- **Visualizing Importance:** The bar chart generated here is a high-value asset for your thesis paper. It grounds your ML model in medical reality.

Common Mistakes

- **Forgetting stratify=y:** If you get incredibly unlucky with a random split, all your CKD patients might end up in the training set, and none in the test set. The model won't know how to evaluate itself.
- **Calling .fit_transform() on the Test Set:** This destroys the validity of your entire project. The test set must only be .transform()ed using the metrics learned from the training set.

Expected Output

The terminal will print the shapes (320 rows for training, 80 for testing). You will then see a horizontal bar chart. You will likely notice that clinical markers like hemoglobin, serum_creatinine, and specific_gravity dominate the top of the chart, confirming that the model is learning valid medical relationships, not random noise.

How to Explain It During Viva

Examiner: "Why did you scale the data *after* splitting it, instead of just scaling the whole CSV file at the start?"

Your Answer: "To prevent Data Leakage. If I scale before splitting, the standard deviation and mean used to scale the training data would be influenced by the test data. The test data must represent future, unseen patients. In a real hospital deployment, the model won't have access to future patients' data to calculate a global mean. It must rely solely on the parameters learned during training."

Run this code. Look closely at the feature importance chart. Let me know which features rank at the very top for you, and we will move into **Phase 5: Baseline Modeling and Algorithm Comparison**.

Phase 5: Baseline Modeling and Model Comparison

This is the most exciting phase of the ML lifecycle. You have done the hard work of preparing the data; now we get to see how well machines can learn from it.

In research, you never train just one model and call it a day. According to the "No Free Lunch" theorem in machine learning, no single algorithm is guaranteed to work best for every dataset. We must test a diverse set of algorithms, establish a "baseline," and compare them objectively.

Phase 5: Baseline Modeling and Model Comparison

Objective

To train a variety of machine learning algorithms on the scaled training data, evaluate them on the unseen test data, and select the top performer based on clinical metrics (specifically Recall).

Theory

We will test four distinct types of algorithms:

1. **Logistic Regression:** A simple, linear baseline. If this performs perfectly, we don't need complex AI.
2. **Support Vector Machine (SVM):** Excellent for finding complex boundaries in high-dimensional spaces.
3. **Random Forest:** An ensemble of decision trees. Very robust to outliers and non-linear data.
4. **Gradient Boosting:** A sequential ensemble technique where each tree corrects the mistakes of the previous one. Often the absolute best performer on tabular datasets.

The Metric Shift:

In this project, we care about **Accuracy, Precision, Recall, and F1-Score**, but **Recall is King**.

- *Precision:* Out of everyone the model *said* had CKD, how many actually had it? (Avoids false alarms).
- *Recall (Sensitivity):* Out of everyone who *actually* has CKD, how many did the model find? (Avoids sending sick people home).

In medical AI, a False Negative (missing a sick patient) is far more dangerous than a False Positive (doing one extra blood test on a healthy patient). Therefore, we will rank our models by Recall and F1-Score.

Implementation: Python Code

Add this block to your Jupyter Notebook and run it.

Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd
```

```
print("Training Baseline Models...\n")
```

```
# 1. Initialize our dictionary of models
```

```
# random_state=42 ensures the algorithms use the same internal random seed for
reproducible research
```

```
models = {
```

```
    "Logistic Regression": LogisticRegression(random_state=42),
```

```
    "Support Vector Machine": SVC(random_state=42),
```

```

    "Random Forest": RandomForestClassifier(random_state=42, n_estimators=100),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

# List to store our evaluation metrics
results = []

# 2. Train and Evaluate each model in a loop
for model_name, model in models.items():
    # Train the model on the training data
    model.fit(X_train_scaled, y_train)

    # Predict on the unseen testing data
    y_pred = model.predict(X_test_scaled)

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Append to our results list
    results.append([model_name, acc, prec, rec, f1])

# 3. Create a Comparison Table
results_df = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall", "F1-Score"])

# Sort the table by Recall (most important for medical), then F1-Score
results_df = results_df.sort_values(by=["Recall", "F1-Score"], ascending=[False, False]).reset_index(drop=True)

print("--- Model Comparison Results ---")
print(results_df.to_string(index=False))

# 4. Visualizing the Comparison
plt.figure(figsize=(10, 6))
# Using pandas built-in plotting for a quick grouped bar chart
results_df.set_index("Model")[["Recall", "Precision", "F1-Score"]].plot(kind='bar',
    colormap='viridis', ax=plt.gca())
plt.title("Model Performance Comparison (Test Set)")
plt.ylabel("Score (0.0 to 1.0)")
plt.xticks(rotation=45, ha='right')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

```

Best Practices

- **Establish a Baseline:** Always include Logistic Regression. If a complex neural network achieves 98% accuracy, but a simple Logistic Regression achieves 97%, the simple model is mathematically "better" because it is computationally cheaper and fully explainable.
- **Fix Random Seeds:** In your code, you will notice `random_state=42` everywhere. Without this, your model's accuracy will change slightly every time you run the code, making it impossible to confidently write a research paper.

Common Mistakes

- **Only looking at Accuracy:** Accuracy can hide fatal flaws in imbalanced datasets.
- **Training on the Test Set:** Notice how we *only* used `X_train_scaled` in the `model.fit()` line.

Expected Output

The terminal will print a very clean, formatted Pandas DataFrame comparing the four models. Because CKD datasets often have strong clinical signal (especially if your KNN imputation from Phase 3 was successful), you might see scores in the very high 90s. One model (often Random Forest or Gradient Boosting) should emerge as the winner with the highest Recall and F1-Score.

How to Explain It During Viva

Examiner: "Why did you choose [Winning Model Name] as your final model instead of the others?"

Your Answer: "I evaluated four algorithms ranging from linear baselines to ensemble methods. While all performed well, I selected the final model based primarily on its Recall score on the unseen test set. In a clinical diagnostic tool for Chronic Kidney Disease, a false negative is medically catastrophic, whereas a false positive merely triggers a secondary screening. [Winning Model Name] minimized false negatives better than Logistic Regression or SVM, while maintaining a strong F1-score to balance precision."

Run the code. Let me know which model achieved the highest Recall and what its F1-Score was! Once we identify our champion model, we will move to **Phase 6: Hyperparameter Tuning**, where we squeeze out the absolute maximum performance.

This is a phenomenal result! Your Random Forest achieved an almost perfect score.

Let's break down exactly what these numbers mean clinically, because in your viva or thesis, *this* table is your "hero graphic."

- **Accuracy (0.9875):** The Random Forest correctly predicted 98.75% of all patients in the unseen test set.
- **Precision (1.000):** Every single time the model predicted "This patient has CKD," it was correct. There were zero false alarms (False Positives = 0).
- **Recall (0.980):** Out of all the patients who *actually* had CKD in the test set, the model successfully identified 98% of them. It missed only a tiny fraction (perhaps 1 patient), which are False Negatives.
- **F1-Score (0.989):** The harmonic mean of Precision and Recall, proving the model is incredibly robust despite any class imbalance.

You might notice Gradient Boosting (which often wins tabular data competitions) scored lowest here. This sometimes happens on smaller datasets (~400 rows) where Random Forest's internal "bagging" mechanism prevents overfitting better than Gradient Boosting's sequential trees.

Key insight: A perfect Precision (1.0) with a 0.98 Recall means your model is slightly conservative. It would rather occasionally miss a sick patient than ever falsely diagnose a healthy one. In a real hospital, we actually want the opposite: we want 1.0 Recall (catch every sick person) even if it means Precision drops to 0.90 (a few healthy people get unnecessary secondary tests).

Phase 6: Hyperparameter Tuning (Squeezing out that last 2%)

Objective

To push the Random Forest's Recall closer to 1.00 by tuning its internal settings (hyperparameters) using GridSearchCV.

Theory

Right now, our Random Forest used default settings (`n_estimators=100` trees). But what if it performs better with 200 trees? What if we limit how deep the trees can grow (`max_depth`) so it doesn't memorize the training data?

We will use **Grid Search Cross-Validation (GridSearchCV)**. It systematically tests hundreds of combinations of these settings, validating them on different "folds" of the training data, and returns the absolute best configuration.

Implementation: Python Code

Add this block to your Jupyter Notebook:

Python

```
from sklearn.model_selection import GridSearchCV
```

```
print("Starting Hyperparameter Tuning for Random Forest...")
```

```
# 1. Define the "Grid" of parameters to test
```

```
# This tells the algorithm to try combinations of these numbers
```

```
param_grid = {
    'n_estimators': [50, 100, 200],      # Number of trees in the forest
    'max_depth': [None, 10, 20, 30],     # Maximum depth of the trees
    'min_samples_split': [2, 5, 10],     # Minimum samples required to split an internal node
    'min_samples_leaf': [1, 2, 4],      # Minimum samples required to be at a leaf node
    'class_weight': ['balanced', None]   # Crucial for medical data: tells the model to care
                                          # equally about both classes
}
```

```
# 2. Initialize the Grid Search
```

```
# We explicitly tell it to score based on 'recall' because that is our medical priority
```

```
rf = RandomForestClassifier(random_state=42)
```

```
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,          # 5-fold cross-validation
    scoring='recall', # Optimize for Recall!
    n_jobs=-1,     # Use all available CPU cores to speed it up
    verbose=1      # Print progress
)
```

```
# 3. Run the Grid Search (This might take 10-30 seconds depending on your machine)
```

```
grid_search.fit(X_train_scaled, y_train)
```

```
# 4. Extract the Best Model
```

```
best_rf_model = grid_search.best_estimator_
```

```
print("\n--- Best Hyperparameters Found ---")
```

```
print(grid_search.best_params_)
```

```
# 5. Evaluate the Tuned Model on the Unseen Test Data
```

```
y_pred_tuned = best_rf_model.predict(X_test_scaled)
```

```
tuned_acc = accuracy_score(y_test, y_pred_tuned)
```

```
tuned_prec = precision_score(y_test, y_pred_tuned)
```

```
tuned_rec = recall_score(y_test, y_pred_tuned)
```

```
tuned_f1 = f1_score(y_test, y_pred_tuned)
```

```
print("\n--- Tuned Random Forest Results ---")
```

```
print(f"Accuracy: {tuned_acc:.4f} (Baseline: {results_df.loc[0, 'Accuracy']:.4f})")
```

```
print(f"Precision: {tuned_prec:.4f} (Baseline: {results_df.loc[0, 'Precision']:.4f})")
```

```
print(f"Recall: {tuned_rec:.4f} (Baseline: {results_df.loc[0, 'Recall']:.4f})")
```

```
print(f"F1-Score: {tuned_f1:.4f} (Baseline: {results_df.loc[0, 'F1-Score']:.4f})")
```

```
# 6. Plot the Final Confusion Matrix
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
cm = confusion_matrix(y_test, y_pred_tuned)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Not CKD (0)', 'CKD (1)'])
```

```
plt.figure(figsize=(6, 6))
```

```
disp.plot(cmap='Blues', values_format='d', ax=plt.gca())
```

```
plt.title("Final Tuned Model Confusion Matrix")
```

```
plt.show()
```

Best Practices

- **Targeting the Scoring Metric:** Setting `scoring='recall'` inside `GridSearchCV` is the mark of a good AI engineer. Default Grid Search optimizes for Accuracy, which we know is dangerous here.
- `class_weight='balanced'`: This parameter forces the Random Forest to penalize itself more heavily if it makes a mistake on the minority class. This often improves Recall significantly.

Common Mistakes

- **Making the Grid too large:** If you add 10 options for every parameter, `GridSearchCV` will test thousands of combinations and your computer will freeze. The grid provided above is a safe, optimized set.

How to Explain It During Viva

Examiner: "Your baseline model already had 98% Recall. Why did you bother with Hyperparameter Tuning?"

Your Answer: "In a university lab, 98% is excellent. In a real hospital, the remaining 2% represents human lives. My goal was to see if optimizing the tree depth (max_depth) or forcing the model to explicitly balance the class weights (class_weight='balanced') could push that Recall to 100% without causing catastrophic degradation in Precision. Furthermore, Grid Search Cross-Validation proves that the model's high performance isn't just a lucky artifact of the default settings."

Run the code! The terminal will print Fitting 5 folds for each of 216 candidates.... Once it finishes, let me know what the **Best Hyperparameters Found** are, and if your Recall hit that magic 1.00. Then we will move to **Phase 7: Explainable AI (SHAP)**.

Do not be disappointed that the scores did not jump to 100%! In machine learning, this is actually a fantastic result.

When your hyperparameter tuning returns the exact same scores as the baseline, it proves two things to your examiners:

1. **Your baseline model was not just "lucky"**. The initial 98% Recall was mathematically stable.
2. **You have reached the "Bayesian Error Rate"** for this specific dataset. The remaining 2% error is likely due to overlapping clinical profiles (a sick patient whose blood work looks perfectly healthy), which no algorithm can perfectly separate without overfitting.

As a researcher, proving that your model is stable is just as important as proving it is accurate.

Now, we move to the phase that separates standard undergraduate projects from publishable research papers.

Phase 7: Explainable AI (XAI) with SHAP

Objective

To crack open the "black box" of our Random Forest model and prove to doctors exactly *why* it diagnosed a specific patient with CKD.

Theory

Medical professionals do not trust an AI that just says "Probability of CKD: 98%." They need to know *what* in the blood work caused that prediction.

We will use **SHAP (SHapley Additive exPlanations)**, which is rooted in cooperative game theory. SHAP calculates the exact mathematical contribution of every single feature (like Hemoglobin or Blood Pressure) to the final prediction. It answers the question: "Did the patient's low hemoglobin push the model toward a CKD diagnosis, and by exactly how much?"

Why It Is Important

Explainability is a massive topic in modern AI research. Adding a SHAP analysis section to your final report or thesis instantly elevates its academic rigor.

Implementation: Python Code

You may need to install the SHAP library first. In a new Jupyter cell, run `!pip install shap`, then run the following code:

Python

```
# Uncomment the line below if you haven't installed shap yet
```

```
# !pip install shap
```

```

import shap

print("Generating SHAP Explainability Plots...")

# 1. Initialize the SHAP TreeExplainer with our best tuned model
# TreeExplainer is highly optimized for Random Forests
explainer = shap.TreeExplainer(best_rf_model)

# 2. Calculate SHAP values for the unseen test data
# This computes the contribution of each feature for every patient in the test set
shap_values = explainer.shap_values(X_test_scaled)

# Note: Random Forest classifiers output SHAP values for both classes [Not CKD, CKD].
# We only want to explain the predictions for the 'CKD' class (index 1).
# (Depending on the shap version, it might return a list or an array. We handle both safely).
if isinstance(shap_values, list):
    shap_values_ckd = shap_values[1]
elif len(shap_values.shape) == 3:
    shap_values_ckd = shap_values[:, :, 1]
else:
    shap_values_ckd = shap_values

# 3. Global Explainability: SHAP Summary Plot
# This shows how features impact the model across ALL patients
plt.figure(figsize=(10, 8))
shap.summary_plot(shap_values_ckd, X_test_scaled, show=False)
plt.title("SHAP Summary Plot: Feature Impact on CKD Prediction")
plt.tight_layout()
plt.show()

# 4. Local Explainability: Explaining a SINGLE Patient's Diagnosis
# Let's look at the very first patient in our test set
patient_index = 0

# Initialize JavaScript for interactive plots in Jupyter
shap.initjs()

# Generate a Force Plot for Patient 0
print(f"\nExplaining prediction for Patient at index {patient_index}...")
print(f"Actual Diagnosis: {'CKD' if y_test.iloc[patient_index] == 1 else 'Not CKD'}")

# The force plot shows features pushing the prediction higher (red) or lower (blue)
shap.force_plot(
    explainer.expected_value[1] if isinstance(explainer.expected_value, (list, np.ndarray)) else
    explainer.expected_value,
    shap_values_ckd[patient_index, :],

```



```
X_test_scaled.iloc[patient_index, :],  
matplotlib=True # Renders as a static image instead of interactive JS (safer for notebooks)  
)
```

Best Practices

- **Global vs. Local Explanation:** Always include both. The **Summary Plot (Global)** proves your model learned valid medical rules (e.g., "Low hemoglobin strongly predicts CKD"). The **Force Plot (Local)** proves your model can justify its diagnosis for an individual patient sitting in the clinic.

Common Mistakes

- **Passing the wrong SHAP values:** A binary classifier outputs SHAP values for *Class 0* and *Class 1*. If you plot the values for Class 0, your chart will look backward (e.g., high hemoglobin causing CKD). Always ensure you extract the values for Class 1 (the positive class).

Expected Output

1. **Summary Plot:** A beautiful dot plot. Red dots indicate high values of a feature, blue dots indicate low values. If hemoglobin is at the top, and all the blue dots are stretched to the right (positive SHAP value), it means "Low hemoglobin heavily increases the chance of CKD."
2. **Force Plot:** A horizontal bar showing the "push and pull" of a single patient's lab results. Red bars push the patient toward a CKD diagnosis; blue bars push them toward being healthy.

How to Explain It During Viva

Examiner: "Your model is highly accurate, but deep learning and random forests are black boxes. How can a doctor trust this?"

Your Answer: "That is exactly why I integrated SHAP. It calculates the marginal contribution of each feature using cooperative game theory. I can provide the clinician with a global summary showing that the model prioritizes Serum Creatinine and Hemoglobin—aligning perfectly with medical consensus. More importantly, for any specific patient, the model generates a Force Plot showing exactly which lab results drove that specific diagnosis, ensuring the AI acts as a transparent second opinion rather than a black box."

Run this code. These plots are the crown jewels of your thesis report. Once you have seen them, we will move to **Phase 8: Model Saving & Deployment Strategy**, where we prepare to turn this Jupyter Notebook into a real application.

This is the transition point where you stop being just a Data Scientist and start becoming a Full-Stack AI Engineer.

A model sitting in a Jupyter Notebook is a research experiment. A model that is saved, exported, and connected to an interface is a *product*.

Phase 8: Model Saving & Deployment Strategy

Objective

To serialize (save) the trained model and every single preprocessing tool we built, so they can be loaded into a web application (like Streamlit, Flask, or FastAPI) without needing to retrain the model every time a user logs in.

Theory (The "Pipeline" Concept)

When a doctor uses your web app, they will type in a patient's raw data: e.g., Blood Pressure = 130, Hypertension = "yes", Red Blood Cells = 5.2.

Your Random Forest model *cannot* understand this. It expects scaled numbers and encoded categories.

Therefore, you cannot just save the model. You must save the **entire pipeline**:

1. **The Label Encoders:** To turn "yes" into 1.
2. **The KNN Imputer:** In case the doctor leaves a field blank.
3. **The Standard Scaler:** To turn blood pressure of 130 into a scaled value like 0.45.
4. **The Trained Model:** To make the actual prediction.
5. **The Feature Order:** To ensure the 24 inputs are fed into the model in the exact sequence it expects.

We use a Python library called joblib to compress and save these Python objects into physical files on your hard drive.

Implementation: Python Code

Create a new folder in your project directory called saved_models (or let the code create it for you). Run this code in a new cell:

Python

```
import joblib
```

```
import os
```

```
print("Exporting Model and Preprocessing Artifacts...")
```

```
# 1. Create a directory to store the files cleanly
os.makedirs('saved_models', exist_ok=True)
```

```
# 2. Save the Best Tuned Model
joblib.dump(best_rf_model, 'saved_models/ckd_random_forest_model.pkl')
```

```
# 3. Save the Scaler
joblib.dump(scaler, 'saved_models/ckd_scaler.pkl')
```

```
# 4. Save the KNN Imputer
joblib.dump(knn_imputer, 'saved_models/ckd_knn_imputer.pkl')
```

```
# 5. Save the Label Encoders (Since it's a dictionary, we save the whole dictionary)
joblib.dump(label_encoders, 'saved_models/ckd_label_encoders.pkl')
```

```
# 6. Save the exactly expected feature column names
# This is crucial for deployment so we know exactly what order the web app should send
data
expected_features = X.columns.tolist()
joblib.dump(expected_features, 'saved_models/ckd_expected_features.pkl')
```

```
print("\nSuccess! All artifacts have been saved to the 'saved_models' folder.")
print("You are now ready to build the web application.")
```

Best Practices

- **Version Control:** In a real company, you would name your file `ckd_rf_model_v1.0.pkl`. If you train a better model next month, you save it as `v1.1`. This allows you to roll back if the new model breaks.
- **Use joblib over pickle:** While pickle is standard Python, joblib is highly optimized for saving scikit-learn models that contain large NumPy arrays.

Common Mistakes

- **Forgetting the Feature Order:** If your Streamlit app sends `blood_pressure` as the first column, but your model was trained with `age` as the first column, the model will output complete garbage predictions, and it *won't throw an error*. Saving the exact list of `expected_features` prevents this.

Expected Output

You should see a new folder named `saved_models` appear in the same directory as your Jupyter Notebook. Inside, there will be 5 `.pkl` (pickle) files. These files are the "brain" of your project, frozen in time.

How to Explain It During Viva

Examiner: "How would you deploy this model in a real clinical setting?"

Your Answer: "I wouldn't deploy just the model; I deploy the entire preprocessing pipeline. I serialized the Random Forest model alongside the `StandardScaler`, `KNNImputer`, and `LabelEncoders` using `joblib`. In a production environment, when a mobile app sends raw patient data via a REST API, my backend loads these exact artifacts. The raw data is encoded, imputed, and scaled using the parameters learned from the original training data before being passed to the model. This guarantees zero data leakage and ensures the model evaluates the new patient under the exact same mathematical conditions it was trained on." Run the code and verify the files are saved.

We are almost at the finish line! The final technical step is **Phase 9: Deployment (Building the Interface)**. For university labs, I highly recommend using **Streamlit**. It allows you to build a beautiful, interactive web app entirely in Python without needing to know HTML or JavaScript.

This is where your hard work becomes a tangible product. A beautiful, functional user interface is what will ultimately impress your non-technical stakeholders (and your professors).

We are going to use **Streamlit**. It is an open-source framework designed specifically for Machine Learning engineers. It allows you to build a full web application using only Python—no HTML, CSS, or JavaScript required.

Phase 9: Deployment (Building the Interface)

Objective

To build a web application that accepts patient data, processes it through our saved pipeline (encoders and scalers), feeds it to the tuned Random Forest model, and displays the diagnosis clearly to a clinician.

Theory

A Machine Learning web app has three distinct layers:

1. **The Frontend (UI):** Sliders, dropdowns, and buttons where the user inputs data.
2. **The Middleware (Preprocessing):** Where the raw input is translated into the scaled, encoded format the model expects.

3. **The Backend (Inference):** Where the model makes the prediction and returns the result.

Why It Is Important

Your algorithm might have 98% Recall, but if a doctor cannot easily input a patient's hemoglobin and blood pressure to get a result, the model is useless. Deployment demonstrates that you are not just a researcher, but a full-stack AI engineer capable of delivering end-to-end solutions.

Implementation: Python Code

You cannot run this inside a Jupyter Notebook cell. You must create a new physical Python file.

1. In your project folder (where saved_models is located), create a new file named app.py.
2. Open your terminal and run pip install streamlit.
3. Paste the following code into app.py:

Python

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
```

1. Page Configuration

```
st.set_page_config(page_title="NephroGuard: CKD Predictor", page_icon="🩺",
layout="wide")
st.title("🩺 NephroGuard: Early-Stage CKD Prediction System")
st.markdown("""
```

This application uses a Machine Learning model to predict the probability of Chronic Kidney Disease (CKD)

based on routine clinical test results.
""")

2. Load the Saved Artifacts

We use st.cache_resource so these large files are only loaded once, making the app fast
@st.cache_resource

```
def load_artifacts():
    model = joblib.load('saved_models/ckd_random_forest_model.pkl')
    scaler = joblib.load('saved_models/ckd_scaler.pkl')
    label_encoders = joblib.load('saved_models/ckd_label_encoders.pkl')
    expected_features = joblib.load('saved_models/ckd_expected_features.pkl')
    return model, scaler, label_encoders, expected_features
```

```
model, scaler, label_encoders, expected_features = load_artifacts()
```

3. Build the User Input Interface

```
st.header("Enter Patient Clinical Data")
```

We will use columns to make the UI look professional and organized

```
col1, col2, col3 = st.columns(3)
```

```

# Define the categorical columns (based on our earlier EDA)
categorical_cols = ['red_blood_cells', 'pus_cell', 'pus_cell_clumps', 'bacteria',
                    'hypertension', 'diabetes_mellitus', 'coronary_artery_disease',
                    'appetite', 'pedal_edema', 'anemia']

# Create a dictionary to store user inputs
user_data = {}

# Programmatically generate input fields based on expected features
for i, feature in enumerate(expected_features):
    # Distribute inputs across the 3 columns
    target_col = col1 if i % 3 == 0 else col2 if i % 3 == 1 else col3

    with target_col:
        if feature in categorical_cols:
            # Reconstruct the original text classes from the label encoder
            original_classes = label_encoders[feature].classes_
            user_data[feature] = st.selectbox(f"{feature.replace('_', ' ').title()}", original_classes)
        else:
            # Default numeric inputs to 0.0
            user_data[feature] = st.number_input(f"{feature.replace('_', ' ').title()}", value=0.0,
            step=0.1)

# 4. Prediction Logic
st.markdown("---")
if st.button("Predict Kidney Disease Risk", type="primary"):

    # A. Convert user dictionary to a Pandas DataFrame
    input_df = pd.DataFrame([user_data])

    # B. Preprocessing: Label Encode the categorical variables
    for col in categorical_cols:
        if col in input_df.columns:
            le = label_encoders[col]
            input_df[col] = le.transform(input_df[col])

    # C. Preprocessing: Scale the numerical variables
    # We must ensure the columns are in the exact same order as training
    input_df = input_df[expected_features]
    input_df_scaled = scaler.transform(input_df)

    # D. Make the Prediction
    prediction = model.predict(input_df_scaled)[0]
    prediction_proba = model.predict_proba(input_df_scaled)[0]

# 5. Display the Results

```

```

st.header("Diagnostic Result")

if prediction == 1:
    st.error(f"⚠️ **High Risk of Chronic Kidney Disease (CKD)** detected.")
    st.write(f"**Model Confidence:** {prediction_proba[1] * 100:.2f}%")
    st.write("Recommendation: Patient requires immediate nephrology consultation.")
else:
    st.success(f"✅ **Low Risk. No CKD detected.**")
    st.write(f"**Model Confidence:** {prediction_proba[0] * 100:.2f}%")
    st.write("Recommendation: Continue standard preventative care.")

```

How to Run Your Web App

1. Save the app.py file.
2. Open your Mac Terminal.
3. Use the cd command to navigate to your project folder (where app.py is located).
4. Run this exact command:

Bash

```
streamlit run app.py
```

5. A local web server will start, and the application will automatically open in your web browser!

Best Practices

- **The @st.cache_resource Decorator:** This is crucial. Without it, Streamlit would reload your Random Forest .pkl file from the hard drive every single time the user clicks a button or types a number. Caching keeps it loaded in RAM, making the app lightning fast.
- **Dynamic UI Generation:** Notice how we looped through expected_features to create the inputs. If you drop a feature in the future to make the model lighter, the UI will automatically adapt without you rewriting the frontend code.

How to Explain It During Viva

Examiner: "Why did you build a web interface instead of just evaluating the model in Jupyter? And how did you handle the data preprocessing in the app?"

Your Answer: "A model confined to a notebook has limited real-world utility. I built a Streamlit application to demonstrate how this system functions in a clinical setting as a diagnostic support tool. Crucially, I maintained strict pipeline integrity: the application captures raw user input, encodes categorical variables using the serialized LabelEncoders from training, and normalizes numeric values using the exact same StandardScaler. This ensures the inference environment mathematically mirrors the training environment, preventing data drift and guaranteeing accurate predictions."

Create the app.py file, run the command in your terminal, and test the UI! Enter some random numbers (e.g., a high Serum Creatinine of 5.0 and low Hemoglobin of 9.0) and see if the app catches the CKD risk.

Once your app is running, let me know. We will wrap up with the **Final Phase: GitHub, Documentation, and Viva Preparation.**

This is an outstanding idea. Adding Optical Character Recognition (OCR) and Information Extraction turns a standard Machine Learning project into an **Advanced Multimodal AI**

System. If you show this in your viva, it demonstrates that you understand the biggest bottleneck in healthcare IT: manual data entry.

To do this, we will integrate **Google's Gemini Vision AI**. It is vastly superior to standard OCR (like Tesseract) because it doesn't just read the text; it understands the medical context and can format the extracted text directly into the structured data your form expects.

The Architecture

1. **Upload:** The user uploads a photo of a medical lab report.
2. **Vision LLM:** We send the image to Gemini with a strict prompt: *"Find the blood pressure, hemoglobin, etc., and return it exactly as a JSON dictionary."*
3. **Session State:** We use Streamlit's `st.session_state` to temporarily save this extracted data.
4. **Auto-Fill:** The UI form automatically populates with the extracted data, allowing the doctor to verify or correct it before clicking "Predict".

Prerequisites

Before updating the code, you need two things:

1. Open your terminal and install the Google Generative AI SDK and Pillow (for image handling):

Bash

```
pip install google-generativeai pillow
```

2. Get a free Gemini API key from [Google AI Studio](#)

. You will need to paste this into the app's sidebar.

Implementation: Updated app.py

Replace your entire `app.py` code with this updated version. I have added comments labeled [NEW] to show exactly what changed.

Python

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
import os
import json
from PIL import Image
import google.generativeai as genai # [NEW] Google Gemini SDK
```

1. Page Configuration

```
st.set_page_config(page_title="NephroGuard: CKD Predictor", page_icon="🩺",
layout="wide")
st.title("🩺 NephroGuard: Early-Stage CKD Prediction System")
st.markdown("""
```

This application uses a Machine Learning model to predict the probability of Chronic Kidney Disease (CKD).

```
    **New Feature:** Upload a clinical lab report image to automatically extract and fill
    patient data!
    """)
```

[NEW] Sidebar for API Key Setup

```
st.sidebar.header("Configuration")
```

```

api_key = st.sidebar.text_input("Enter Gemini API Key (for Image Extraction)",
type="password")
if api_key:
    genai.configure(api_key=api_key)

# 2. Load the Saved Artifacts
@st.cache_resource
def load_artifacts():
    current_dir = os.path.dirname(os.path.abspath(__file__))
    model = joblib.load(os.path.join(current_dir, 'saved_models',
'ckd_random_forest_model.pkl'))
    scaler = joblib.load(os.path.join(current_dir, 'saved_models', 'ckd_scaler.pkl'))
    label_encoders = joblib.load(os.path.join(current_dir, 'saved_models',
'ckd_label_encoders.pkl'))
    expected_features = joblib.load(os.path.join(current_dir, 'saved_models',
'ckd_expected_features.pkl'))
    return model, scaler, label_encoders, expected_features

model, scaler, label_encoders, expected_features = load_artifacts()

# [NEW] Initialize Streamlit Session State for Extracted Data
# Streamlit refreshes the script on every click. We must save extracted data in session_state.
if 'extracted_data' not in st.session_state:
    st.session_state.extracted_data = {}

# [NEW] Image Upload & Extraction Section
st.header("📄 Step 1: Auto-Fill from Lab Report (Optional)")
uploaded_file = st.file_uploader("Upload Lab Report Image (JPG/PNG)", type=["jpg", "jpeg",
"png"])

if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Lab Report", width=400)

if st.button("Extract Data with AI"):
    if not api_key:
        st.error("Please enter your Gemini API Key in the sidebar first.")
    else:
        with st.spinner("Analyzing medical report..."):
            try:
                # Define exactly what we want Gemini to extract
                # We pass the list of expected features so it knows the exact keys to use
                prompt = f"""
                You are a medical data extraction assistant.
                Analyze this clinical lab report and extract the following clinical metrics:
                {expected_features}.
            """

```


Rules:

1. If a value is missing or cannot be found, DO NOT include it in the output.
2. For categorical features (like hypertension, diabetes_mellitus), extract them as 'yes', 'no', 'normal', 'abnormal', 'present', 'notpresent', 'good', 'poor' based on the report.
3. Return ONLY a raw JSON object (no markdown, no backticks, no explanation).
Example format: {"blood_pressure": 120.0, "sugar": 0.0, "hypertension": "no"}

```
# Call Gemini 1.5 Flash (fast and supports vision)
```

```
vision_model = genai.GenerativeModel('gemini-1.5-flash')
```

```
response = vision_model.generate_content([prompt, image])
```

```
# Clean the response (sometimes the LLM wraps JSON in markdown block)
```

```
raw_text = response.text.strip().removeprefix('```json').removesuffix('```').strip()
```

```
extracted_dict = json.loads(raw_text)
```

```
# Save to session state
```

```
st.session_state.extracted_data = extracted_dict
```

```
st.success("Data extracted successfully! Review the populated form below.")
```

```
except Exception as e:
```

```
st.error(f"Extraction failed: {e}")
```

```
# 3. Build the User Input Interface
```

```
st.header("📋 Step 2: Verify & Enter Patient Data")
```

```
col1, col2, col3 = st.columns(3)
```

```
categorical_cols = ['red_blood_cells', 'pus_cell', 'pus_cell_clumps', 'bacteria',  
                    'hypertension', 'diabetes_mellitus', 'coronary_artery_disease',  
                    'appetite', 'pedal_edema', 'anemia']
```

```
user_data = {}
```

```
for i, feature in enumerate(expected_features):
```

```
    target_col = col1 if i % 3 == 0 else col2 if i % 3 == 1 else col3
```

```
# [NEW] Check if AI extracted a value for this feature to use as default
```

```
extracted_val = st.session_state.extracted_data.get(feature, None)
```

```
with target_col:
```

```
    if feature in categorical_cols:
```

```
        original_classes = list(label_encoders[feature].classes_)
```

```
# Determine the default index if extracted
```

```
default_index = 0
```

```
if extracted_val is not None:
```

```
    extracted_val = str(extracted_val).lower().strip()
```

```

        if extracted_val in original_classes:
            default_index = original_classes.index(extracted_val)

        user_data[feature] = st.selectbox(f"{feature.replace('_', ' ').title()}", original_classes,
index=default_index)
    else:
        # Determine default numeric value
        default_num = 0.0
        if extracted_val is not None:
            try:
                default_num = float(extracted_val)
            except ValueError:
                pass # Fallback to 0.0 if extraction was weird text

        user_data[feature] = st.number_input(f"{feature.replace('_', ' ').title()}",
value=default_num, step=0.1)

```

4. Prediction Logic

```

st.markdown("---")
if st.button("Predict Kidney Disease Risk", type="primary"):

```

```

    input_df = pd.DataFrame([user_data])

```

```

    for col in categorical_cols:
        if col in input_df.columns:
            le = label_encoders[col]
            input_df[col] = le.transform(input_df[col])

```

```

    input_df = input_df[expected_features]
    input_df_scaled = scaler.transform(input_df)

```

```

    prediction = model.predict(input_df_scaled)[0]
    prediction_proba = model.predict_proba(input_df_scaled)[0]

```

5. Display the Results

```

st.header("Diagnostic Result")

```

```

if prediction == 1:
    st.error(f"⚠️ **High Risk of Chronic Kidney Disease (CKD)** detected.")
    st.write(f"**Model Confidence:** {prediction_proba[1] * 100:.2f}%")
else:
    st.success(f"✅ **Low Risk. No CKD detected.**")
    st.write(f"**Model Confidence:** {prediction_proba[0] * 100:.2f}%")

```

Best Practices Introduced Here

- **Human-in-the-Loop:** We don't automatically make a prediction the moment the image is scanned. AI can hallucinate or misread messy handwriting. We use the AI to

fill the form, allowing the doctor (the human) to verify the data before clicking predict. This is a crucial safety standard in medical software.

- **Session State:** Notice the if 'extracted_data' not in st.session_state: block. When you click a button in Streamlit, it reruns the whole script from top to bottom. If we didn't save the extracted dictionary in the session state, it would disappear the moment the UI refreshed.

Run the updated code. Grab any fake blood test report image from Google Images, upload it, and click Extract! Let me know when you have tested it.